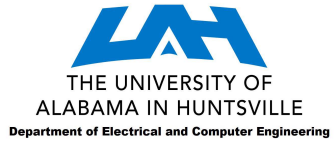



# CPE 324 Advanced Logic Design Laboratory

## Laboratory Assignment #6

## Temperature Sensor

(8 % of Final Grade)

## Purpose

The purpose of this laboratory project is to provide you with the opportunity to develop an interface module in Verilog that can drive an external commercially-off-the-shelf Integrated circuit. This module, in turn, is to be driven and monitored within a larger digital system. To allow for experimental testing of your interface module this larger system will take the form an FPGA-based physical testbed. The modules within this testbed have all been modeled using the VHDL hardware description language. Thus this project gives you the opportunity to experience adding a component to a multi-hardware description language design capture environment. The temperature sensor interface module that you are to develop is to be designed in a manner that will wake up the DS1620 temperature sensor out of its low power consumption mode allowing it to perform its analog to digital temperature acquisition and conversion. Your interface should then be able to receive this temperature using the DS1620 3-wire proprietary serial protocol. Once this has occurred, your interface module should output the received temperature in parallel form, so it can be used by one or more other modules in the system.

To experimentally verify your DS1620 interface in the lab, it will be driven by a physical testbench. The physical testbench will be used to supply the clock and triggering signals to your interface which will allow it to drive an external DS1620 IC. The physical testbench will also monitor the temperature output from your DS1620 interface module and display this temperature using the seven segment LEDs that are present on the Terasic DE2-115 Educational Board. The physical testbench has been designed so that it can display this temperature using either Fahrenheit or the Celsius temperature scales.

Before experimentally verifying your design, using the physical testbench, you will be required to debug it and verify its functionality through digital logic simulation using Questa. This is to be done within the Quartus, design flow ecosystem. The Quartus design flow, allows you to enter your design using Quartus but then switch back and forth between Quartus and Questa when debugging your modeling code. To accomplish this, a standard simulation testbench will be used to drive the portions of the file that you are creating. Quartus facilitates the incorporation of such simulation test benches as a normal part of your design flow process.

## Expected Outcomes

- You will become more proficient at using and obtain a practical knowledge in using the Quartus/Questa design-flow ecosystem.
- From Quartus, you will be better able to utilize the Questa digital logic simulation tool to aid you in interactively debug and verify your portion of the design.
- You will learn how to integrate Verilog modules into a larger digital design that has been constructed using another Hardware Description Design paradigm (VHDL).
- To do this integration, you will gain specific knowledge about the VHDL entity/architecture concepts and how to instantiate Verilog modules from VHDL.
- By replacing one on the VHDL modules with a Verilog one, you will also learn the specifics of how to instantiate VHDL modules from Verilog.
- You will experimentally verify that your design works with actual DS1620 IC hardware using a physical testbench that is implement on the Terasic DE2-115 rapid prototyping platform.

## Physical Testbench Environment

The ds1620_interface module is to drive the DS1620 IC in the following manner: All temperature readings are to be accurate to plus or minus 0.5 degrees Celsius and to be tested using the Teraasic DE2-115 Rapid Prototyping Board which should act as a physical testbench. The testbench should trigger the temperature acquisition process for one temperature reading on the ds1620_interface module whenever you quickly press and release the specified KEY[3] button on DE2-115 board or it should continuously trigger it whenever the slide switch SW[1] is switched to the up position. Switching slide switch SW[0] up and down will toggle the output format between Fahrenheit and Celsius.

The level 1 block diagram for the physical testbench design is shown in Figure 1. In the actual design, this level is implemented in structural VHDL. The *BINTOHEX*, *TEMP2BCD*, *CLOCK_DIVIDE* modules are all modeled as components (i.e. modules) in VHDL. The clock division module, *CLOCK_DIVIDE*, is used to slow down the system clock so that it could be used in an indirect manner to clock the DS1620 IC. In this same high-level design this engineer had also identified the Altera Cyclone IV E inputs and output pins that are to be connected to the DS1620 pins, internal 50 MHZ clock, and seven segment display LEDs.

The module that is to be implemented in Verilog is the *ds1620_interface* module that you will designed. This is the module that that directly interfaces with the DS1620 chip in a manner that allows it to convert the temperature into a digital data format and communicate this temperture data to the physical test bench. This module itself is clocked by the divided clock signal and drives the DS 1620's DQ bidirectional signal (using the built-in tristate pin buffer), RST input, and the DS 1620 CLK input.

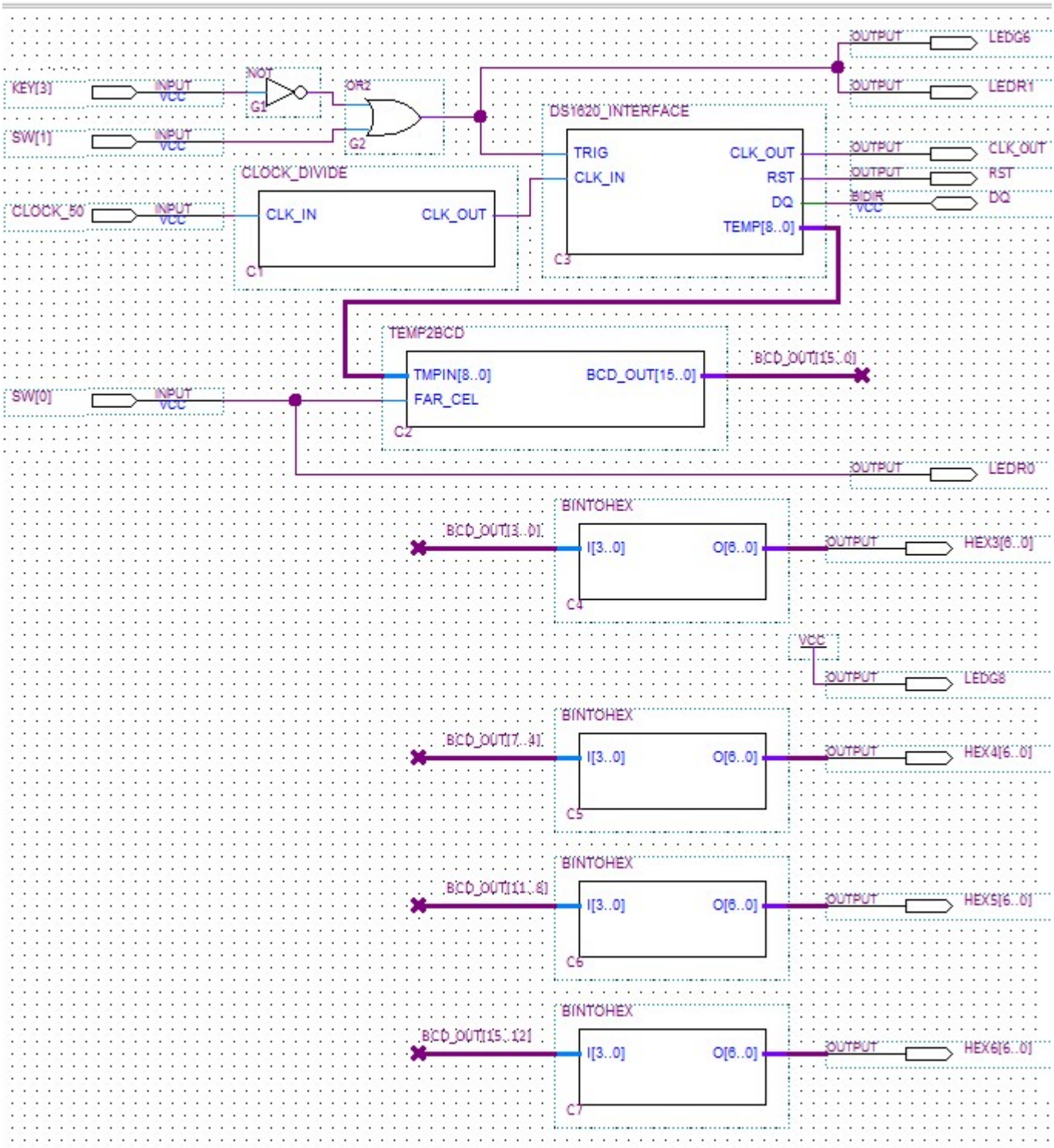



**Figure 1. Level 1 Structural Block-level Representation of Temperature Sensor's Physical Testbench Configuration**

The prototype implementation of the temperature sensor design is to be implemented on the Altera DE2-115 Educational board. It is to utilize the GPIO 0 interface port which is connected to the DS 1620 IC in the manner shown below in Table 1

| Signal Name | Cyclone IV E Pin Number | DE2-115 GPIO_0 Pin | UAH ECE DE2-115 Breakout board Pin | DS 1620 Pin Number | DS 1620 Pin Name |
|---|---|---|---|---|---|
| VCC | | 11 | 3V3 | 8 | VDD |
| GND | | 12 | GND | 4 | GND |
| DQ | PIN_AE20 | 14 | D30 | 1 | DQ |
| CLK_OUT | PIN_AF20 | 32 | D32 | 2 | CLK/$\overline{\text{CONV}}$ |
| RST | PIN_AH23 | 16 | D34 | 3 | $\overline{\text{RST}}$ |

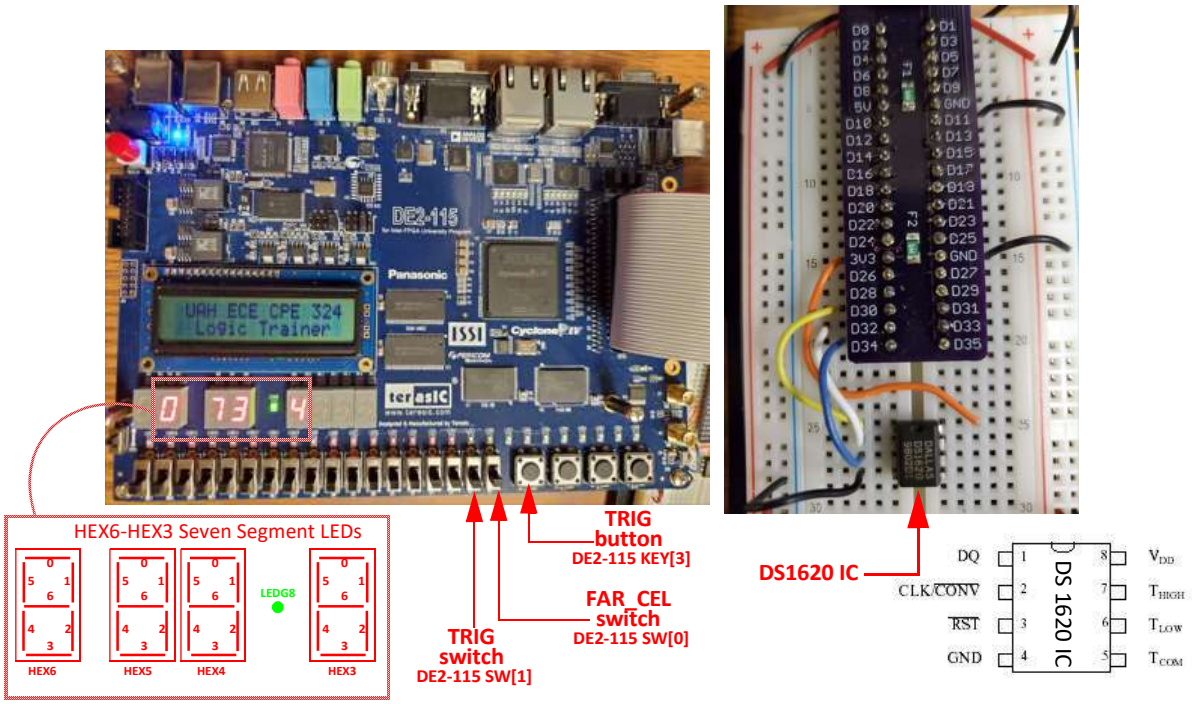



**Table 2: 50 Mhz Internal Clock and Switches on the DE2-115 Board**

| Signal Name | Cyclone IV E Pin Number | Description |
|---|---|---|
| CLK_IN | PIN_Y2 | 50 MHz clock input |
| FAR_CEL | PIN_AB28 | DE2-115 Toggle Switch SW[0] -- Fahrenheit when down, Celsius when up |
| TRIG | PIN_R24 | Push button Key Switch KEY[3] -- Trigger temp conversions when pressed, stop when released |
| TRIG | PIN_AC28 | DE2-115 Toggle Switch SW[1] -- Trigger temp conversions continuously on when in up position |

**Table 3: Cyclone IV E Pin Numbers for HEX6-HEX3 7-Seg LEDs on DE2-115 Board**

| Segment | HEX6 | | HEX5 | | HEX4 | | HEX3 | |
|---|---|---|---|---|---|---|---|---|
| | Signal Name | Cyclone IV E Pin No. | Signal Name | Cyclone IV E Pin No. | Signal Name | Cyclone IV E Pin No. | Signal Name | Cyclone IV E Pin No. |
| 0 | HEX6[0] | PIN_AA17 | HEX5[0] | PIN_AD18 | HEX4[0] | PIN_AB19 | HEX3[0] | PIN_V21 |
| 1 | HEX6[1] | PIN_AB16 | HEX5[1] | PIN_AC18 | HEX4[1] | PIN_AA19 | HEX3[1] | PIN_U21 |
| 2 | HEX6[2] | PIN_AA16 | HEX5[2] | PIN_AB18 | HEX4[2] | PIN_AG21 | HEX3[2] | PIN_AB20 |
| 3 | HEX6[3] | PIN_AB17 | HEX5[3] | PIN_AH19 | HEX4[3] | PIN_AH21 | HEX3[3] | PIN_AA21 |
| 4 | HEX6[4] | PIN_AB15 | HEX5[4] | PIN_AG19 | HEX4[4] | PIN_AE19 | HEX3[4] | PIN_AD24 |
| 5 | HEX6[5] | PIN_AA15 | HEX5[5] | PIN_AF18 | HEX4[5] | PIN_AF19 | HEX3[5] | PIN_AF23 |
| 6 | HEX6[6] | PIN_AC17 | HEX5[6] | PIN_AH18 | HEX4[6] | PIN_AE18 | HEX3[6] | PIN_Y19 |

## The *ds1620_interface* module

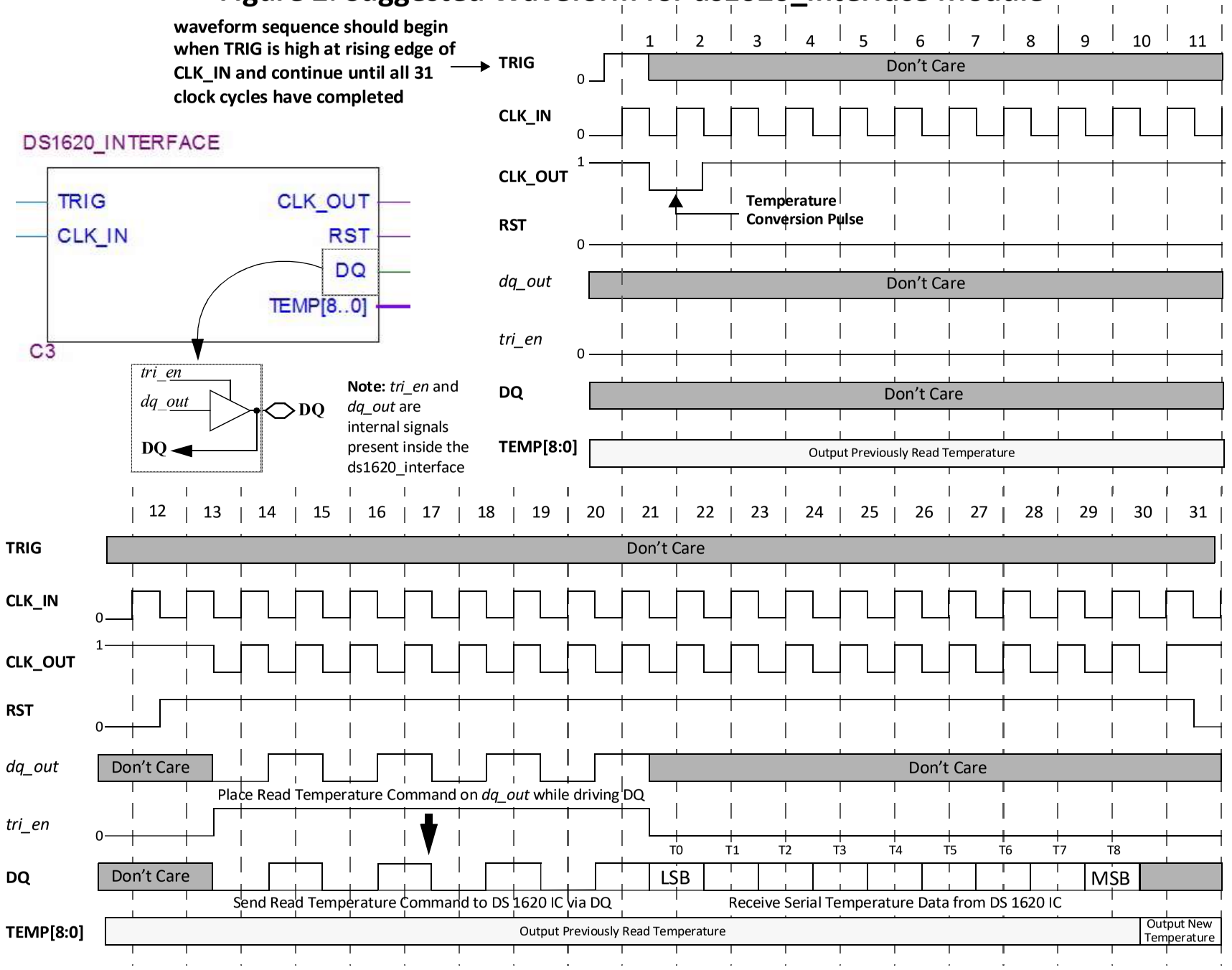
Figure 2 illstrates the suggested waveform that should be produced by the ds1620_interface module to trigger temperture conversions on the DS1620 and to communicate and decode the temperature data measured on the DS1620 back to the interface module. This wavefore will cause the DS 1620 IC be run in its stand-alone thermostat mode where a single negative strobe of the CLK/CONV pin of the DS 1620 will cause it to perform a current temperature measurement/conversion. The waveform then causes the temperature to be read from the DS 1620 using the so called "three wire" mode that is discussed in the DS 1620 Data Sheet. In this mode, the DQ pin must first be driven in a serial manner by the interface circuitry that is being designed. Upon receiving the read temperature command, the direction of the DQ pin will then change from an output to an input and the DS 1620 will drive the DQ pin in a serial manner to output the 9 bit temperature reading. The complete protocol for the three wire operation is described in detail in the DS 1620 data sheet. To simplify matters for this assignment, a detailed timing diagram has been created. This timing diagram is shown in Figure 2. This waveform in this timing is composed of 31 cycles and begins after the first rising edge of the divided system clock after the trigger input signal, TRIG, is placed in a logic high before the rising edge of the CLK_IN signal. Once started this way the entire 31 cycle pattern shown in Figure 1 should occur and the TRIG signal is not monitored again until the cycle patter is complete.

Shown below is the stub file that you are to modify for the *ds1620_interface* module:

```
// Verilog  Module to input temperature data from the DS 1620
// temperature IC
// template by
// B. Earl Wells, ECE Department University of Alabama in Huntsville
// inputs:
//   TRIG an active high Trigger input which execute 31 clock cycle
//      temperature conversion/temperature reading cycle.
//      This cycle is composed of three parts:
//        1) initiate temperature conversion by sending a ~conv pulse
//           when the DS 1620 is in its stand-alone mode
//        2) wait 10 cycles while DS 1620 in stand-alone mode for
//           temperature conversion to take place
//        3) initiate 3-wire communication mode of the DS 1620 and
//           issue read temperature command to DS1620 and receive
//           the 9 bit temperature from it. After which return
//           DS1620 to stand-alone mode
//
//   CLK_IN the input clock (must be less than 1.7 MHZ)
//
// outputs:
//   TEMP the temperature acquired from the DS 1620 in 9 bit two's
//      complement format. (Output Resolution is 1/2 degree C)
//   CLK_OUT the clocking signal that drives the DS 1620 during its
//      "three wire" communication as well as providing the start
//      conversion pulse when the DS 1620 is in the stand-alone mode
//      that causes the DS 1620 to perform the next temperature
//      conversion
//   RST the reset control line of the DS 1620 which is high during
//      "three wire" communication but low for stand-alone mode
//   TRI_EN the controling signal for the external tristate buffer.
//      This signal indicates when the DQ input of the external
//      DS 1620 will be driven by DQ_OUT (this occurs when TRI_EN='1')
//      or when the DS 1620 will drive the DQ_IN input (this occurs
```

```
//       when TRI_EN='0')
//    DQ  the inout signal that is used to input temperature data produce by
//       the DS 1620 IC and outputs command data to the 1620 when the
//       Interface is in 3 wire mode. Note that the input data received from
//       the 1620 IC is always an immediate response to a command issued
//       the interface. When data is sent from the DS 1620, it is in a
//       bit serial manner (least significant bit first). Such
//       data is assumed to be valid at the rising edge of the CLK_OUT
//       signal. In this design, the DQ signal is used to receive
//       the 9 bit Celsius temperature from the DS 1620 after it has
//       received the "read temperature" command from the ds1620_interface.

module ds1620_interface(input TRIG, CLK_IN,
   output CLK_OUT,RST,inout DQ, output reg [8:0] TEMP);

   // dq_out: this internal signal is used to drive the DQ
   //     bidirectional inout signal of the ds1620 interface
   //     during "three wire" communication. Communication using
   //     this signal occurs is synchronized closely with the
   //     CLK_OUT signal. Such communication is bit serial with
   //     with data being sent least significant bit first and
   //     the valid logic value of each bit being placed on the
   //     the dq_out line at least 50 ns before the rising edge
   //     edge of CLK_OUT. Note in this design, dq_out is used to
   //     send the read temperature command to the DS 1620.
   reg dq_out;

   // tri_en: this internal signal is used to enable the tri-state
   //      buffer that allows the internal dq_out signal to drive
   //      the DQ inout signal
   reg tri_en=0;

   // one way to selectively echo the CLK_IN signal to the CLK_OUT
   // signal is to employ a continuos assignment statement that
   // first logically OR's the CLK_IN signal with the OR mask
   // value and then AND's this result with an AND mask. The AND Mask
   // will always force the CLK_OUT signal low if it is low. If the AND mask is
   // high and the OR mask low then the original CLK_IN Value is passed
   // to CLK_OUT unabated. If the AND mask is a high and the OR mask is low
   // then CLK_OUT will always be high. The key here is to set these masks
   // appropriately at strategic points in the waveform.
   reg and_mask = 1, or_mask = 1;

   reg [4:0] cycle_count = 5'd0; // clock cycle count
   reg [8:0] next_temp;         // next temperature value
   reg rst;

   // Enter your model for the ds1620_interface HERE!
   // enter your modeling code here!

endmodule
```

# Figure 2: Suggested Waveform for ds1620_interface Module



## Assignment

This laboratory assignment is composed of three phases as shown below. You must successfully complete and demonstrate the valid functionality of a given phase of your design to the lab instructor before proceeding to the next phase. Demonstration will involve simulation using Questa and/or configuring the DE2-115 board to incorporate your design within the physical test bench. To facilitate design entry a template file named ***lab6_template.zip*** is provided on the course Canvas website. This template should be first downloaded into your personal file space on the PC's in the laboratory such as your desktop and unzipped. When this is done all the files and subdirectories should be present under the *lab6* folder. These files include the Quartus project file which includes the pin assignments as well as the top-level file, component files, as well as the stub *ds1620_interface.v* file that are to be modify.

- Phase I: Creation and Verification through Questa Simulation of the ***ds1620_Interface.v*** Module.
- Phase 2: Verification of the *ds1620_module* using the Physical Testbench
- Phase 3: Manual Creation, Integration and Verification of New Level 1 Verilog version of the physical testbench module, ***lab6_ptb.v***

Each student is to work individually and demonstrate the design to the laboratory instructor. The final report should conform to the previously released standard and include documented Verilog code for the *ds1620_interface* module, simulation results. Also in your report you are to indicate whether the design worked as expected the first time the ds1620_interface module was inserted or if further debugging was required before it worked correctly. If this is the case then indicate what issue or issues were present that were not detected by simulation.

## Post-Lab Questions

1. In your report describe in detail what each component in the physical testbench does.
2. Rewrite a functionally equivalent CLOCK_DIVIDE module in Verilog so that it that could interface directly with the rest of the testbench design.

This project is already two months behind schedule and your supervisor has assigned you the task of completing the design. She has imposed an absolute deadline of April 16, 2024 for you to develop a complete functioning prototype and to fully document the design. You need to complete and demonstrate this project during this time period -- the final round of lay-offs has been rumored for next month. It should be noted that the physical testbench modules have all been written in VHDL not Verilog! This is ok though because you know that with the version of the Quartus tool you can integrate your Verilog module into the VHDL design and will not have to convert that portion of the design the Verilog. This is being done by an intern and should be complete very soon! But you need to work concurrently.

The assignment for this class laboratory includes

1) The design and complete documentation of the *ds1620_interface* module

2) Verification output from the simulation of the *ds1620_interface* design using Questa.

3) A complete description of the operation of the Temperature Sensor design shown in Figure 1.

4) The manually created Verilog replacement for your level 1 ....

## Practical IP Core Notes

To facilitate your design effort, a structural VHDL model of the schematic diagram of the physical testbench is shown in Figure 1. The file *lab6_ip_core_elements.vhd* contains the top-level design VHDL entity and architecture and all the supporting IP core VHDL entity/architecture elements. You are to supply the *ds1620_interface* module which should be written in Verilog. A template file for the ds1620_interface module is also present it is in a file that is named ds1620_interface.v. To access these files download the *lab6_template.zip* file from Canvas and extract it to a location that you have permission to access. Then enter all the pin assignments for the design. These are contained in Tables 1-3 and then compile the design again.

In order to function correctly with the VHDL entered components, the ds1620_interface module you create should list the port signals in the order shown in the verilog template that was provided which is shown below:

## Background Futuristic Design Scenario

For this laboratory you are to take on the role of a entry-level design engineer who is working for a U. S. based manufacturing firm who is responsible for product development of a stand-alone, internet agnostic, temperature sensor device.

Recent litegation

Generative AI that was trained using G

GitHub and other repositories were inappropriately used when they were used to train the first generation of generative AI engines.

Training data was inappropriately purchased from web-scrapping companies thereby violating patent, copywrites, IP property rights

The task at hand is to develop the interface electronics to implement a temperature sensor that is to continuously display the current temperature in Fahrenheit and Celsius. The original intent of this design was to create a low-cost module to directly replace legacy technology which could be easily integrated into multiple products of varying purposes using the latest high-density integrated circuit technology.

Unfortunately, right after you were hired by the company, the supply chain access to high-density foundries by U.S. registered companies was curtailed by the host countries due to a geopolitical international event. This has resulted in an economic downturn which in turn caused a "temporary" reduction in force for your company. This reduction in force has trickled down to

your own design team and has caused the project to be re-scoped.

The goal now, is to use your product to retrofit many legacy temperature monitoring configurations using complex programmable logic devices. The interface electronics now is to be designed using dedicated application specific logic as opposed to a microcontroller base logic, and is to interface to two seven segment LEDs or LCDs. Because of the massive stockpiles of Microchip (Dallas Semiconductor) DS1620 integrated circuit module that have been discovered in company inventory you are to direct all development efforts to create the digital electronics that will effectively interface to these ICs.

Before his employment contract was terminated during the last round of lay-offs at the company, the person who was responsible for the temperature sensor design had created